

Bài 1: TỔNG QUAN VỀ CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Mục tiêu:

Về kiến thức:

- + Nắm được các bước trong lập trình để giải quyết cho một bài toán
- + Nắm vững các khái niệm liên quan đến cấu trúc dữ liệu và giải thuật
- + Nắm vững kỹ thuật đệ quy trong cách giải bài toán
- + Nắm được tư tưởng một số thuật toán sắp xếp và tìm kiếm cơ bản

Về kỹ năng:

- + Vận dụng các tiêu chuẩn để lựa chọn cấu trúc dữ liệu tốt cho bài toán
- + Sử dụng được một số thuật toán cơ bản

1.1 Từ bài toán đến chương trình

Để giải một bài toán trong thực tế bằng máy tính ta thường trải qua 2 giai đoạn:

Giai đoạn 1:

Xác định bài toán cần giải quyết và xây dựng mô hình toán học cho bài toán. Mục đích của giai đoạn này là nhằm trả lời hai câu hỏi: Bài toán cho cái gì và yêu cầu làm những gì? sau đó trả lời câu hỏi “để thực hiện các yêu cầu của bài toán thì làm như thế nào”

Giai đoạn 2:

Cài đặt chương trình: Sử dụng ngôn ngữ lập trình cụ thể để viết chương trình tương ứng với cách làm của giai đoạn trước nó

1.1.1 Xác định và mô hình hóa bài toán cần giải quyết

Khi giải quyết 1 bài toán thực tế, ta phải bắt đầu từ việc xác định bài toán. Nhiều thời gian và công sức bỏ ra để xác định bài toán cần giải quyết, tức là phải trả lời rõ ràng các câu hỏi bài toán yêu cầu ta "phải làm những công việc gì?" sau đó là để thực hiện các công việc này thì ta "làm như thế nào?". Thông thường, khi khởi đầu, hầu hết các bài toán là không đơn giản, không rõ ràng. Để giảm bớt sự phức tạp của bài toán thực tế, ta phải

hình thức hóa nó, nghĩa là phát biểu lại bài toán thực tế thành một bài toán hình thức (hay còn gọi là mô hình toán). Có thể có rất nhiều bài toán thực tế có cùng một mô hình toán.

Ví dụ 1: Xét bài toán tô màu bản đồ thế giới.

Ta cần phải tô màu cho các nước trên bản đồ thế giới. Trong đó mỗi nước đều được tô một màu và hai nước láng giềng (cùng biên giới) thì phải được tô bằng hai màu khác nhau. Hãy tìm một phương án tô màu sao cho số màu sử dụng là ít nhất.

Việc lựa chọn và xây dựng mô hình toán học cho bài toán diễn ra theo phân tích sau:

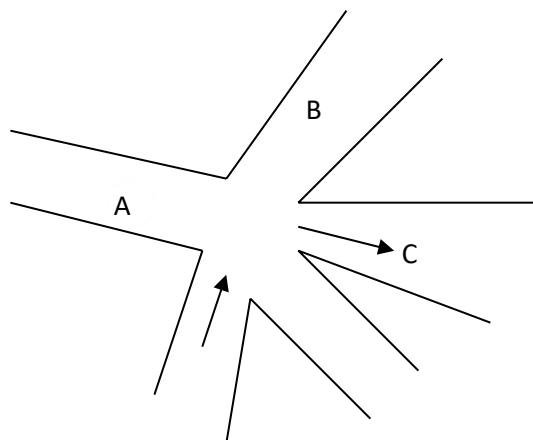
Ta có thể xem mỗi nước trên bản đồ thế giới là một đỉnh của đồ thị, hai nước láng giềng của nhau thì hai đỉnh ứng với nó được nối với nhau bằng một cạnh. Bài toán lúc này trở thành bài toán tô màu cho đồ thị như sau:

- Mỗi đỉnh đều phải được tô màu,
- Hai đỉnh có cạnh nối thì phải tô bằng hai màu khác nhau và ta cần tìm một phương án tô màu sao cho số màu được sử dụng là ít nhất.

=> Mô hình toán học được sử dụng trong bài toán này là mô hình đồ thị.

Ví dụ 2: Xét bài toán điều khiển đèn giao thông

Cho một ngã năm như **hình 1.1**, trong đó C và E là các đường một chiều theo chiều mũi tên, các đường khác là hai chiều. Hãy thiết kế một bảng đèn hiệu điều khiển giao thông tại ngã năm này một cách hợp lý, nghĩa là: phân chia các lối đi tại ngã năm này thành các nhóm, mỗi nhóm gồm các lối đi có thể cùng đi đồng thời nhưng không xảy ra tai nạn giao thông (các lối đi này có các hướng đi không cắt nhau), và số lượng nhóm chia là ít nhất có thể được.

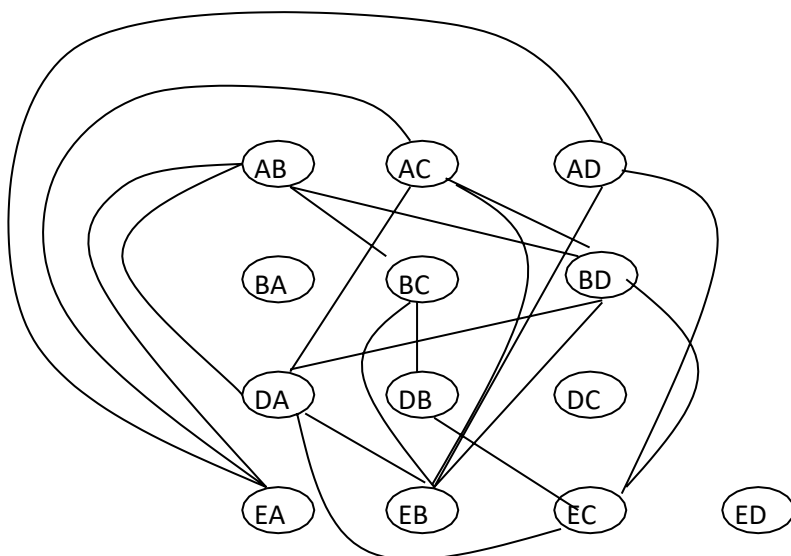


Hình 1.1: Mô hình bài toán điều khiển đèn giao thông

Việc lựa chọn, xây dựng mô hình toán học cho bài toán được diễn ra theo phân tích sau:

Ta có thể xem đầu vào của bài toán là tất cả các lối đi tại ngã năm này, đầu ra của bài toán là các nhóm lối đi có thể đi đồng thời mà không xảy ra tai nạn giao thông, mỗi nhóm sẽ tương ứng với một pha điều khiển của đèn hiệu, vì vậy ta phải tìm kiếm lời giải với số nhóm là ít nhất để giao thông không bị tắc nghẽn vì phải chờ đợi quá lâu.

Trước hết ta nhận thấy rằng tại ngã năm này có 13 lối đi: AB, AC, AD, BA, BC, BD, DA, DB, DC, EA, EB, EC, ED. Tất nhiên, để có thể giải được bài toán ta phải tìm một cách nào đó để thể hiện mối liên quan giữa các lối đi này. Lối nào với lối nào không thể đi đồng thời, lối nào và lối nào có thể đi đồng thời. Ví dụ cặp AB và EC có thể đi đồng thời, nhưng AD và EB thì không, vì các hướng giao thông cắt nhau. Ở đây ta sẽ dùng một sơ đồ trực quan như sau: tên của 13 lối đi được viết lên mặt phẳng, hai lối đi nào nếu đi đồng thời sẽ xảy ra đụng nhau (tức là hai hướng đi cắt qua nhau) ta nối lại bằng một đoạn thẳng, hoặc cong, hoặc ngoằn ngoèo tùy thích. Ta sẽ có một sơ đồ như **hình 1.2**. Như vậy, trên sơ đồ này, hai lối đi có cạnh nối lại với nhau là hai lối đi không thể cho đi đồng thời.



Hình 1.2: Sơ đồ lối đi bài toán điều khiển đèn giao thông

Với cách biểu diễn như vậy ta đã có một mô hình toán học đồ thị (Graph), trong đó mỗi lối đi trở thành một đỉnh của đồ thị, hai lối đi không thể cùng đi đồng thời được nối nhau bằng một đoạn ta gọi là cạnh của đồ thị. Bây giờ ta phải xác định các nhóm, với số nhóm ít nhất, mỗi nhóm gồm các lối đi có thể đi đồng thời, nó ứng với một pha của đèn hiệu điều khiển giao thông. Giả sử rằng, ta dùng màu để tô lên các đỉnh của đồ thị này sao cho:

Các lối đi cho phép cùng đi đồng thời sẽ có cùng một màu: Dễ dàng nhận thấy rằng hai đỉnh có cạnh nối nhau sẽ không được tô cùng màu.

Số nhóm là ít nhất: ta phải tính toán sao cho số màu được dùng là ít nhất. Tóm lại, ta phải giải quyết bài toán "Tô màu" cho đồ thị ở **hình 1.2** sao cho:

Hai đỉnh có cạnh nối với nhau (hai còn gọi là hai đỉnh kề nhau) không cùng màu.

Số màu được dùng là ít nhất."

=> Trở về bài toán tương tự ví dụ trước

Như vậy:

Cả hai bài toán trên, ban đầu có vẻ rất khác nhau, nhưng sau khi phân tích để hình thức hóa thì chúng đều được đưa về mô hình toán học đồ thị, và áp dụng thuật toán tô màu trên đồ thị để giải quyết các bài toán này.

Chú ý: Có rất nhiều các cấu trúc toán học có thể làm mô hình dữ liệu trong tin học, ví dụ như: Danh sách, tập hợp, ánh xạ, cây, đồ thị, ...

+ Một bài toán thực tế bất kỳ thường bao gồm các đối tượng dữ liệu và các yêu cầu xử lý trên những đối tượng đó, cho nên trong giai đoạn phân tích và thiết kế, khi xây dựng mô hình toán học cho bài toán cần chú trọng đến hai vấn đề:

+ Tổ chức biểu diễn các đối tượng dữ liệu của bài toán trong mô hình toán học như thế nào? mô hình này ta còn gọi là mô hình dữ liệu trong tin học ?

+ Xây dựng các thao tác xử lý trên các đối tượng của mô hình ra sao?

1.1.2 Cài đặt chương trình cho bài toán cần giải quyết

Khi cài đặt chương trình giải quyết bài toán tương ứng ta quan tâm đến hai vấn đề:

Biểu diễn mô hình dữ liệu/mô hình toán học của bài toán trên máy tính

Ta phải chọn dạng biểu diễn nào để máy tính có thể hiểu và thực hiện các thao tác trên chúng. Giai đoạn này còn được gọi là xây dựng cấu trúc dữ liệu cho bài toán. Ta có thể cài đặt một mô hình dữ liệu bởi nhiều cấu trúc dữ liệu khác nhau. Trong mỗi cách cài đặt, một số phép toán trên mô hình có thể được thực hiện thuận lợi, nhưng các phép toán khác có thể lại không thuận lợi.

Mã hóa các giải thuật

Với mỗi cấu trúc dữ liệu ta cần mã hóa các thao xử lý trên nó để giải quyết các yêu cầu đặt ra của bài toán.

Ta có thể sử dụng một ngôn ngữ lập trình cụ thể nào đó (Pascal, C, ...) để cài đặt kết quả ở giai đoạn *Xác định & mô hình hóa bài toán cần giải quyết*, ở bước này ta dùng các cấu trúc dữ liệu được cung cấp trong ngôn ngữ, ví dụ Array, Record, ... để biểu diễn mô hình dữ liệu của bài toán trên máy tính, và mã hóa giải thuật bởi các câu lệnh trong ngôn ngữ lập trình lựa chọn.

Như vậy, ta có thể tóm tắt các bước từ bài toán đến chương trình như sau:

Về mặt dữ liệu:

Theo quy trình từ: Mô hình dữ liệu -> Kiểu dữ liệu trừu tượng -> Cấu trúc dữ liệu. Thật vậy: Trong quá trình phát triển chương trình, nhất là khi phát triển các hệ thống phần mềm lớn, ta cần đến hai dạng biểu diễn dữ liệu: Biểu diễn trừu tượng và biểu diễn cụ thể:

Trong giai đoạn xác định và mô hình hóa bài toán: ta cần sử dụng dạng biểu diễn trừu tượng: được xác định bởi mô hình dữ liệu – đó là mô hình toán học của các đối tượng dữ liệu cùng với các phép toán thực hiện trên các đối tượng đó, ví dụ như: Mô hình cây, danh sách, tập hợp, đồ thị, mô hình ERA, ... Khi ta dùng mô hình dữ liệu với một số xác định các phép toán nào đó, ta sẽ có một kiểu dữ liệu trừu tượng, ví dụ: Ngăn xếp,

hàng đợi, bảng băm,....

=> *Dạng biểu diễn dữ liệu này không phụ thuộc vào ngôn ngữ lập trình cụ thể*

Trong giai đoạn cài đặt chương trình, ta cần sử dụng dạng biểu diễn cụ thể của dữ liệu: Là biểu diễn xác định cách lưu trữ vật lý của dữ liệu trong bộ nhớ máy tính.

Biểu diễn cụ thể của dữ liệu được xác định bởi các cấu trúc dữ liệu. Các cấu trúc dữ liệu được mô tả trong ngôn ngữ lập trình cụ thể mà ta sử dụng.

=> *Dạng biểu diễn này phụ thuộc vào ngôn ngữ lập trình cụ thể*

Từ biểu diễn trừu tượng, ta có thể chuyển dịch thành các biểu diễn cụ thể khác nhau, hay nói cách khác, từ các mô hình dữ liệu hoặc từ các kiểu dữ liệu trừu tượng, ta có thể chuyển dịch thành các cấu trúc dữ liệu khác nhau. Ví dụ, ta có thể cài đặt danh sách bởi cấu trúc dữ liệu mảng hoặc cấu trúc dữ liệu danh sách liên kết. Khi cài đặt mô hình dữ liệu bởi cấu trúc dữ liệu nào đó, thì các phép toán trên mô hình được thực hiện bởi các thao tác là cần thiết phải được cài đặt trên cấu trúc dữ liệu đó.

Về mặt xử lý dữ liệu:

Theo quy trình từ: Giải thuật không hình thức -> giải thuật bằng ngôn ngữ giả

-> Giải thuật được mã hóa hoàn toàn bởi ngôn ngữ lập trình cụ thể, ví dụ: Pascal, C,...Thật vậy: Từ những yêu cầu xử lý thực tế, ta tìm các giải thuật trên mô hình dữ liệu đã xây dựng. Giải thuật có thể mô tả một cách không hình thức (tức là nó chỉ nêu phương hướng giải hoặc các bước giải một cách tổng quát). Tiếp theo ta hình thức hoá giải thuật bằng ngôn ngữ giả, rồi chi tiết hoá dần ("mịn hóa") các bước giải tổng quát ở trên (làm mịn dần), ở bước này ta cần dùng các kiểu dữ liệu trừu tượng (không phải là các khai báo cài đặt trong ngôn ngữ lập trình cụ thể) và các cấu trúc lệnh điều khiển trong ngôn ngữ lập trình (không chú trọng đến cú pháp ngôn ngữ), kết hợp ngôn ngữ tự nhiên để mô tả giải thuật. Cuối cùng trong pha cài đặt, ta tiến hành mã hóa hoàn toàn giải thuật được mô tả bởi ngôn ngữ giả, sử dụng ngôn ngữ lập trình cụ thể, thao tác trên cấu trúc dữ liệu cụ thể.

1.2 Các khái niệm cơ bản

1.2.1 Xác định và mô hình hóa bài toán cần giải quyết

Định nghĩa: Mô hình dữ liệu là mô hình toán học cùng với các phép toán có thể thực hiện trên các đối tượng của mô hình. Mô hình dữ liệu được sử dụng để mô tả cấu trúc logic của dữ liệu được xử lý bởi hệ thống.

Lựa chọn mô hình dữ liệu cho bài toán cần giải quyết như thế nào?

Các thành phần dữ liệu thực tế đa dạng, phong phú và thường chứa đựng những quan hệ nào đó với nhau, do đó cần phải tổ chức, lựa chọn và xây dựng các mô hình dữ liệu thích hợp nhất sao cho vừa có thể phản ánh chính xác các dữ liệu thực tế này, vừa có thể dễ dàng dùng máy tính để xử lý.

Để tìm ra cấu trúc toán học thích hợp với một bài toán đã cho, trước hết chúng ta cần phải phân tích kỹ bài toán để tìm ra câu trả lời cho các câu hỏi sau:

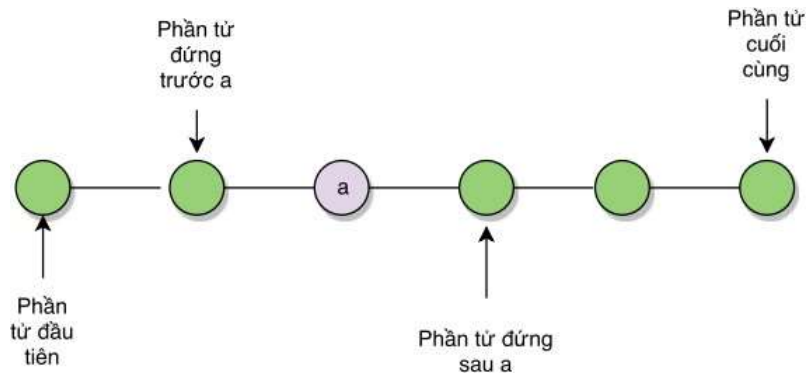
- + Các thông tin quan trọng của bài toán có thể biểu diễn bởi các đối tượng toán học nào?
- + Có các mối quan hệ nào giữa các đối tượng?
- + Các kết quả phải tìm của bài toán có thể biểu diễn bởi các khái niệm toán học nào?

Sau khi đã có mô hình toán học mô tả bài toán, một câu hỏi đặt ra là, ta phải làm việc với mô hình như thế nào để tìm ra lời giải của bài toán? Chúng ta sẽ thiết kế các thuật toán thông qua các hành động, các phép toán thực hiện trên các đối tượng của mô hình.

Các loại mô hình dữ liệu

Ta có thể phân loại các mô hình dữ liệu dựa trên mối quan hệ giữa các phần tử của chúng như sau:

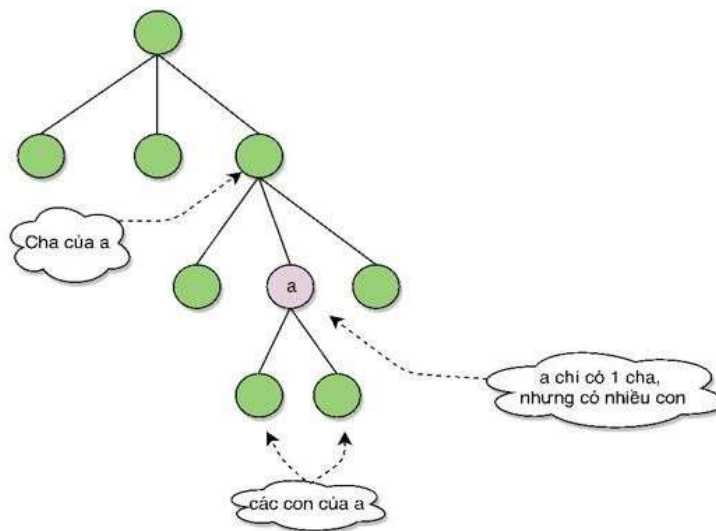
Mô hình dữ liệu tuyến tính (danh sách): Dùng để biểu diễn các phần tử có quan hệ 1:1. Các phần tử trong mô hình có quan hệ tuyến tính theo thứ tự xuất hiện của chúng, tức là, nếu mô hình dữ liệu tuyến tính chứa các phần tử thì nó phải có phần tử đầu tiên và phần tử cuối cùng, mỗi phần tử có đúng một phần tử đứng ngay trước và một phần tử đứng ngay sau. **Hình 1.3** biểu diễn một ví dụ về mô hình dữ liệu tuyến tính.



Hình 1.3: Mô hình dữ liệu danh sách

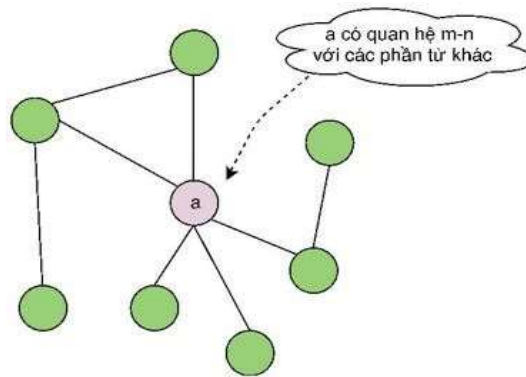
Mô hình dữ liệu phân cấp (mô hình cây): Dùng để biểu diễn các phần tử có quan hệ 1: n, tức là mỗi phần tử trong mô hình có nhiều hậu bối, nhưng chỉ có một tiền bối.

Hình 1.4 biểu diễn một ví dụ cụ thể về mô hình này. Nếu ta di chuyển từ trên xuống dưới trong **Hình 1.4** thì mỗi nút có thể trở đến nhiều nút khác, nhưng nếu ta di chuyển từ dưới lên thì mỗi nút (trừ nút ở gốc) chỉ có quan hệ với 1 nút. Mô hình dữ liệu phân cấp như vậy thường được gọi là cây, và đây là một loại mô hình dữ liệu quan trọng trong khoa học máy tính:



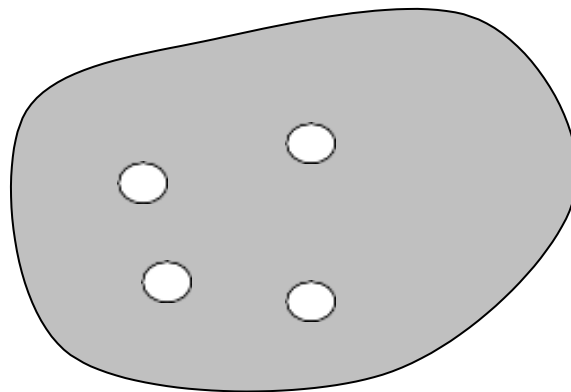
Hình 1.4: Mô hình dữ liệu cây

Mô hình dữ liệu đồ thị: đây là mô hình dữ liệu phong phú và phức tạp nhất. Trong đồ thị, các phần tử có mối quan hệ $n:m$. Tức là, mỗi phần tử có thể có quan hệ với một hoặc nhiều phần tử khác. Hình 1.5 biểu diễn một ví dụ cụ thể về mô hình này.



Hình 1.5: Mô hình dữ liệu đồ thị

Mô hình dữ liệu tập hợp. Trong một tập hợp, các phần tử không có mối quan hệ trực tiếp với nhau, giữa chúng chỉ có một mối quan hệ là thành viên của tập hợp, ta không cần quan tâm tới vị trí chính xác của một phần tử nào đó trong tập hợp. **Hình 1.6** biểu diễn một ví dụ cụ thể về mô hình này.



Hình 1.6: Mô hình dữ liệu tập hợp

Trên đây là bốn loại mô hình dữ liệu mà chúng ta sẽ nghiên cứu trong cuốn bài giảng này. Ta cũng sẽ nghiên cứu các dạng biểu diễn của các mô hình này bởi các cấu trúc dữ liệu khác nhau trong pha cài đặt chương trình. Nói chung, hầu hết các cấu trúc dữ liệu đều rơi vào một trong bốn dạng cơ bản này.

1.2.2 Khái niệm trừu tượng hóa

Trong tin học, trừu tượng hóa nghĩa là đơn giản hóa, làm cho nó sáng sủa hơn và dễ hiểu hơn. Cụ thể trừu tượng hóa là che đi những chi tiết, làm nổi bật cái tổng thể.

1.2.3 Kiểu dữ liệu trừu tượng

Trong Mô hình dữ liệu, chúng ta có thể thực hiện một tập hợp các phép toán rất đa dạng, phong phú. Song trong nhiều áp dụng, chúng ta chỉ sử dụng mô hình với một số xác định các phép toán nào đó. Khi đó chúng ta sẽ có một kiểu dữ liệu trừu tượng

Kiểu dữ liệu trừu tượng (abstract data type): là một mô hình dữ liệu được xét cùng với một số xác định các phép toán nào đó.

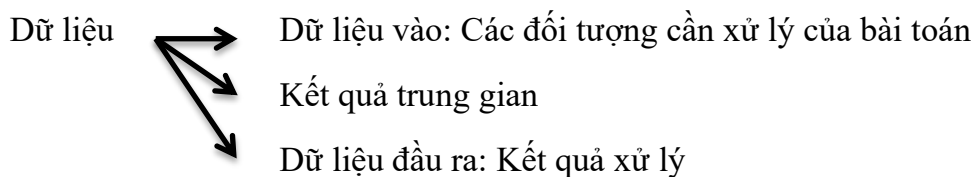
Ví dụ: Mô hình dữ liệu danh sách, chỉ xét đến các phép toán thêm vào và lấy ra, ta gọi là kiểu dữ liệu hàng đợi hoặc ngăn xếp, Mô hình tập hợp, chỉ xét đến các phép toán: Thêm vào, loại bỏ, tìm kiếm ta gọi là kiểu dữ liệu trừu tượng từ điển,

Khi nói đến một kiểu dữ liệu cần phải đề cập đến hai đặc trưng cơ bản sau :

- 1) Tập các giá trị thuộc kiểu.
- 2) Tập hợp các phép toán có thể thực hiện được trên các dữ liệu của kiểu.

1.2.4 Dữ liệu

Thực tế dữ liệu tồn tại ở rất nhiều dạng: hình ảnh, âm thanh,... có rất nhiều dạng khác nhau. Trong một bài toán, dữ liệu được phân làm ba loại:



Ví dụ: Xét bài toán cho một dãy số gồm n số nguyên, tìm số lớn nhất trong dãy số. Các dữ liệu của bài toán là:

- Dữ liệu đầu vào: số nguyên n , dãy số $a_1, a_2, \dots, a_n$
- Dữ liệu đầu ra: Số lớn nhất (max)
- Dữ liệu trung gian: Giả sử dùng biến điều khiển i trong quá trình duyệt từ đầu đến cuối dãy để tìm số lớn nhất.

1.2.5 Biểu diễn dữ liệu trên máy tính

Trong máy tính, các dữ liệu dù tồn tại ở những hình thức khác nhau (số, văn bản, hình ảnh, đúng/sai...) đều được biểu diễn dưới dạng nhị phân khi đưa vào MT xử lý. Tức là mỗi dữ liệu được biểu diễn dưới dạng một dãy các số nhị phân 0 hoặc 1.

Ví dụ: Số 10 = 0000 1010 ở dạng nhị phân

=> Cách biểu diễn này rất không thuận tiện (dài, khó, không gợi nhớ, ...) đối với con người. Việc xuất hiện các ngôn ngữ lập trình bậc cao (Pascal, C, ... gần với ngôn ngữ tự nhiên) đã giải phóng con người khỏi những khó khăn khi làm việc với cách biểu diễn dữ liệu nhị phân trong MT

Trong các ngôn ngữ lập trình bậc cao: Các kiểu dữ liệu là sự trừu tượng hoá các tính chất của các đối tượng dữ liệu có cùng bản chất trong thế giới thực (chỉ ra những tính chất đặc trưng cho các đối tượng thuộc phạm vi bài toán đang xét)

Ví dụ: Ứng với các dữ liệu dạng số, tương ứng ta có các kiểu dữ liệu số nguyên, số thực, số phức, ... trong ngôn ngữ lập trình,

Như vậy tất cả các dữ liệu mô tả trong ngôn ngữ lập trình bậc cao được máy tính xử lý đều phải được khai báo thuộc một kiểu dữ liệu xác định. Khi đó chương trình dịch của ngôn ngữ lập trình bậc cao sẽ dịch chuyển các kiểu dữ liệu này sang dạng biểu diễn nhị phân trước khi được máy tính xử lý.

1.2.6 Kiểu dữ liệu

Mỗi kiểu dữ liệu trong ngôn ngữ lập trình tương ứng với một loại dữ liệu trong thực tế. Ví dụ chúng ta có kiểu dữ liệu số nguyên: Đại diện cho tất cả các đối tượng dữ liệu là số nguyên, tất cả các dữ liệu được xử lý trong chương trình đều phải thuộc kiểu dữ liệu xác định được khai báo trong ngôn ngữ lập trình.

- Trong ngôn ngữ lập trình, một kiểu dữ liệu **T** được xác định bởi một bộ **<V,O>** trong đó:

V : tập các giá trị hợp lệ mà một đối tượng kiểu **T** có thể lưu trữ **O** : tập các thao tác xử lý có thể thi hành trên đối tượng kiểu **T**. Ví dụ: Giả sử có kiểu dữ liệu **ký tự (Character)** = **<V_c,O_c>** với **V_c = { a-z, A-Z }**

O_c = {lấy mã ASCII của ký tự, biến đổi ký tự thường thành ký tự hoa...}

Giả sử có kiểu dữ liệu **số nguyên** = **<V_i,O_i>** với

V_i = { -32768..32767 } **O_i = { +, -, *, /, % }**

Như vậy:

Muốn sử dụng một kiểu dữ liệu trong cài đặt cần nắm vững các thuộc tính của kiểu dữ liệu đó. Các thuộc tính của 1 kiểu dữ liệu bao gồm:

- Tên kiểu dữ liệu: Từ khóa thể hiện cho kiểu đó
- Miền giá trị: Một biến có kiểu dữ liệu đó có thể nhận các giá trị trong phạm vi nào
- Kích thước lưu trữ: Để tối ưu hóa việc sử dụng kiểu dữ liệu phù hợp, tránh hiện tượng dư thừa bộ nhớ.
- Tập các toán tử tác động lên kiểu dữ liệu: Các phép toán cơ bản mà kiểu dữ liệu đó cung cấp.

Ta thấy rằng, các dữ liệu cơ bản thường là các loại dữ liệu đơn giản, không có cấu trúc. Chúng thường là các giá trị vô hướng như các số nguyên, số thực, các ký tự, các giá trị logic ... Các loại dữ liệu này, do tính thông dụng và đơn giản của mình, thường được các ngôn ngữ lập trình (NNLT) cấp cao xây dựng sẵn như một thành phần của ngôn ngữ để giảm nhẹ công việc cho người lập trình. Chính vì vậy đôi khi người ta còn gọi chúng là các kiểu dữ liệu định sẵn/tiền định. Thông thường, các kiểu dữ liệu cơ bản bao gồm :

- Kiểu có thứ tự rời rạc: số nguyên, ký tự, logic , liệt kê, miền con ...
- Kiểu không rời rạc: số thực

Tùy ngôn ngữ lập trình, các kiểu dữ liệu định nghĩa sẵn có thể khác nhau đôi chút về các thuộc tính. Ví dụ: Với ngôn ngữ C, các kiểu dữ liệu này chỉ gồm số nguyên, số thực, ký tự. Và theo quan điểm của C, kiểu ký tự thực chất cũng là kiểu số nguyên về mặt lưu trữ, chỉ khác về cách sử dụng. Trong khi đó PASCAL định nghĩa tất cả các kiểu dữ liệu cơ sở đã liệt kê ở trên và phân biệt chúng một cách chặt chẽ.

Các kiểu cơ sở rất đơn giản và không thể hiện rõ sự tổ chức dữ liệu trong một cấu trúc, thường chỉ được sử dụng làm nền để xây dựng các kiểu dữ liệu phức tạp khác.

Trong nhiều trường hợp, chỉ với các kiểu dữ liệu cơ sở không đủ để phản ánh tự nhiên và đầy đủ bản chất của sự vật thực tế, dẫn đến nhu cầu phải xây dựng các kiểu dữ liệu mới dựa trên việc tổ chức, liên kết các thành phần dữ liệu có kiểu dữ liệu đã được định nghĩa. Những kiểu dữ liệu được xây dựng như thế gọi là kiểu dữ liệu có cấu trúc. Đa số các ngôn ngữ lập trình đều cài đặt sẵn một số kiểu có cấu trúc cơ bản như mảng, chuỗi, tập tin, bản ghi... và cung cấp cơ chế cho lập trình viên tự định nghĩa kiểu dữ liệu mới.

Ví dụ : Để mô tả một đối tượng sinh viên, cần quan tâm đến các thông tin sau:

- Mã sinh viên: chuỗi ký tự
- Tên sinh viên: chuỗi ký tự
- Ngày sinh: kiểu ngày tháng
- Nơi sinh: chuỗi ký tự
- Điểm thi: số nguyên

Các kiểu dữ liệu cơ sở cho phép mô tả một số thông tin như: Diemthi: integer;

Các thông tin khác đòi hỏi phải sử dụng các kiểu có cấu trúc như:

- Masv: string[15];
- Tensv: string[15];
- Noisinh: String[15];

Để thể hiện thông tin về ngày tháng năm sinh cần phải xây dựng một kiểu dữ liệu có cấu trúc bản ghi.

```
// định nghĩa cấu trúc ngày sinh:
typedef struct Date{
    int ngay;
    int thang;
    int nam;
};
```

Cuối cùng, ta có thể xây dựng kiểu dữ liệu có cấu trúc thể hiện thông tin về một sinh viên:

```
// định nghĩa cấu trúc sinh viên
typedef struct Sinhvien{
    char maSV[15];
    char tenSV[15];
    char noiSinh[25];
    int diemThi;
    struct Date ngaySinh;
};
```

Giả sử đã có cấu trúc phù hợp để lưu trữ một sinh viên, nhưng thực tế lại cần quản lý nhiều sinh viên, lúc đó nảy sinh nhu cầu xây dựng kiểu dữ liệu có cấu trúc mới, ví dụ danh sách hoặc mảng...

Mục tiêu của việc nghiên cứu cấu trúc dữ liệu chính là tìm những phương cách thích hợp để tổ chức, liên kết dữ liệu, hình thành các kiểu dữ liệu có cấu trúc từ những kiểu dữ liệu đã được định nghĩa.

1.2.7 Cấu trúc dữ liệu

Cấu trúc dữ liệu là tập hợp các thành phần dữ liệu khác nhau kết hợp trong một thể thống nhất. Cấu trúc dữ liệu được định mô tả như sau: **CTDL = {Các dữ liệu thành phần} (1)**

Trong đó: Các dữ liệu thành phần có thể là dữ liệu đơn (sẵn có) hoặc là CTDL đã được xây dựng, chúng được liên kết với nhau theo một phương pháp liên kết nào đó, từ những CTDL này người ta xây dựng các giải thuật tương ứng tác động trên CTDL đó một cách hiệu quả nhất. Ví dụ: Trong ngôn ngữ lập trình Pascal, các cấu trúc dữ liệu đã được định nghĩa sẵn như:

Mảng: Bao gồm một dãy có thứ tự các phần tử có cùng 1 kiểu dữ liệu xác định nào đó.

Bản ghi: Bao gồm một tập các phần tử có thể thuộc các kiểu dữ liệu khác nhau, có mối quan hệ với nhau đó là chúng cùng mô tả các thuộc tính của một đối tượng, ví dụ thông tin về 1 con người gồm họ tên, ngày sinh, chiều cao, cân nặng, ...

Theo cách định nghĩa như trên: (1) là cách mô tả cấu trúc dữ liệu trực tiếp nhất, ngoài ra người ta có thể định nghĩa cấu trúc dữ liệu thông qua các khái niệm khác như: Cấu trúc dữ liệu là kiểu dữ liệu được biểu diễn trên máy tính hoặc Cấu trúc dữ liệu là kiểu dữ liệu có cấu trúc.

Để đánh giá một Cấu trúc dữ liệu người ta thường căn cứ vào các tiêu chuẩn sau:

Phản ánh đúng thực tế: Đây là tiêu chuẩn quan trọng nhất, quyết định tính đúng đắn của toàn bộ bài toán. Cần xem xét kỹ lưỡng cũng như dự trù các trạng thái biến đổi của dữ liệu trong chu trình sống để có thể chọn cấu trúc dữ liệu lưu trữ thể hiện chính xác đối tượng thực tế.

Ví dụ: Một số tình huống chọn cấu trúc lưu trữ sai: Chọn một biến số nguyên integer để lưu trữ tiền thưởng bán hàng (được tính theo công thức tiền thưởng bán hàng = trị giá hàng * 5%), do vậy sẽ làm tròn mọi giá trị tiền thưởng gây thiệt hại cho nhân viên bán hàng. Trường hợp này phải sử dụng biến số thực để phản ánh đúng kết quả của công thức tính thực tế. Trong trường trung học, mỗi lớp có thể nhận tối đa 28 học sinh. Lớp hiện có 20 học sinh, mỗi tháng mỗi học sinh đóng học phí 100.000đ. Chọn một biến số nguyên byte (khả năng lưu trữ 0 - 255) để lưu trữ tổng học phí của lớp học trong tháng là không phù hợp vì giá trị tổng học phí thu được > 255, vượt khỏi khả năng lưu trữ của biến đã chọn, gây ra tình trạng tràn, dẫn đến sai lệch.

Phù hợp với các thao tác trên đó: Tiêu chuẩn này giúp tăng tính hiệu quả của giải thuật, cụ thể là giúp cho việc phát triển các thuật toán đơn giản, tự nhiên hơn trên cấu trúc. Do đó chương trình đạt hiệu quả cao hơn về tốc độ xử lý.

Ví dụ : Một tình huống chọn cấu trúc lưu trữ không phù hợp:

Cần xây dựng một chương trình soạn thảo văn bản, các thao tác xử lý thường xảy ra là chèn, xóa sửa các ký tự trên văn bản. Trong thời gian xử lý văn bản, nếu chọn cấu trúc

lưu trữ văn bản trực tiếp lên tập tin thì sẽ gây khó khăn khi xây dựng các giải thuật cập nhật văn bản và làm chậm tốc độ xử lý của chương trình vì phải làm việc trên bộ nhớ ngoài. Trường hợp này nên tìm một cấu trúc dữ liệu có thể tổ chức ở bộ nhớ trong để lưu trữ văn bản suốt thời gian soạn thảo.

LƯU Ý: Đối với mỗi ứng dụng, cần chú ý đến thao tác nào được sử dụng nhiều nhất để lựa chọn cấu trúc dữ liệu cho thích hợp

Tiết kiệm tài nguyên hệ thống: Cấu trúc dữ liệu chỉ nên sử dụng tài nguyên hệ thống vừa đủ để đảm nhiệm được chức năng của nó. Thông thường có 2 loại tài nguyên cần lưu tâm nhất: CPU và bộ nhớ. Tiêu chuẩn này nên cân nhắc tùy vào tình huống cụ thể khi viết chương trình. Nếu cần một chương trình có tốc độ xử lý nhanh thì khi chọn cấu trúc dữ liệu yếu tố tiết kiệm thời gian xử lý phải đặt nặng hơn tiêu chuẩn sử dụng tối ưu bộ nhớ, và ngược lại.

Ví dụ : Một số tình huống chọn cấu trúc lưu trữ lãng phí:

- + Sử dụng biến **integer** (2 bytes) để lưu trữ một giá trị cho biết tháng hiện hành . Biết rằng tháng chỉ có thể nhận các giá trị từ 1-12, nên chỉ cần sử dụng kiểu **byte** là đủ.

- + Để lưu trữ danh sách học viên trong một lớp, sử dụng mảng 50 phần tử (giới hạn số học viên trong lớp tối đa là 50). Nếu số lượng học viên thật sự ít hơn 30, thì gây lãng phí. Trường hợp này cần có một cấu trúc dữ liệu linh động hơn mảng, ví dụ danh sách liên kết – ta sẽ đề cập đến trong các bài tiếp theo.

Như vậy: Kiểu dữ liệu phức/có cấu trúc, hay còn gọi là cấu trúc dữ liệu, là kiểu dữ liệu trong đó các phần tử của nó có thể phân tách thành các kiểu dữ liệu đơn hoặc kiểu dữ liệu phức khác. Ví dụ, kiểu dữ liệu phức trong Pascal bao gồm: *array* và *record*.

1.2.8 Giải thuật

Khi đã có mô hình toán học thích hợp cho một bài toán ta cần cố gắng tìm cách giải quyết bài toán trong mô hình đó. Khởi đầu là tìm một giải thuật, đó là một chuỗi hữu hạn các chỉ thị (instruction) mà mỗi chỉ thị có một ý nghĩa rõ ràng tương ứng với một thao tác và thực hiện được trong một lượng thời gian hữu hạn.

Knuth (1973) định nghĩa *giải thuật* là *một chuỗi hữu hạn các thao tác để giải một bài toán nào đó*. Các tính chất quan trọng của giải thuật là:

– **Hữu hạn (finiteness):**

Giải thuật phải luôn luôn kết thúc sau một số hữu hạn bước.

– **Xác định (definiteness):**

Mỗi bước của giải thuật phải được xác định rõ ràng và phải được thực hiện chính xác, nhất quán.

– **Hiệu quả (effectiveness):**

Các thao tác trong giải thuật phải được thực hiện trong một lượng thời gian hữu hạn. Ngoài ra một giải thuật còn phải có đầu vào (input) và đầu ra (output).

Tóm lại: Một giải thuật phải giải quyết xong công việc khi ta cho dữ liệu vào. Có nhiều cách để thể hiện giải thuật: dùng ngôn ngữ tự nhiên, dùng lưu đồ, dùng ngôn ngữ giả, dùng các câu lệnh của ngôn ngữ lập trình,... Và một cách dùng phổ biến là dùng ngôn ngữ giả, đó là sự kết hợp của ngôn ngữ tự nhiên và các câu lệnh của ngôn ngữ lập trình.

1.2.9 Mối quan hệ giữa cấu trúc dữ liệu và giải thuật

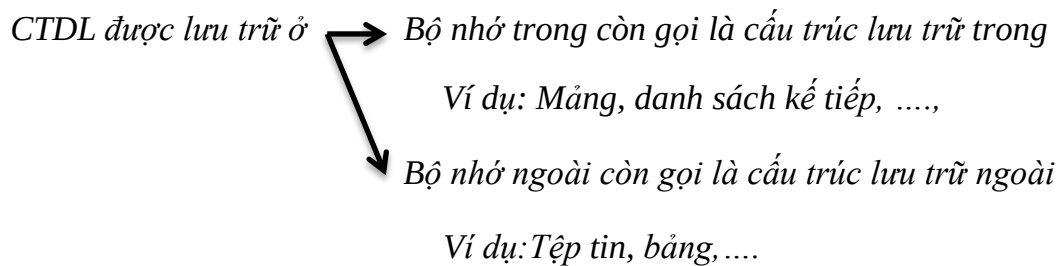
Trong một chương trình: Giải thuật phản ánh các phép xử lý, còn đối tượng xử lý của giải thuật lại là dữ liệu, chính dữ liệu chứa đựng các thông tin cần thiết để thực hiện giải thuật. Để xác định được giải thuật phù hợp cần phải biết nó tác động đến loại dữ liệu nào (ví dụ để làm nhuyễn các hạt đậu, người ta dùng cách xay chứ không băm bằng dao, vì đậu sẽ văng ra ngoài) và khi chọn lựa cấu trúc dữ liệu cũng cần phải hiểu rõ những thao tác nào sẽ tác động đến nó (ví dụ để biểu diễn các điểm số của sinh viên người ta dùng số thực thay vì chuỗi ký tự vì còn phải thực hiện thao tác tính trung bình từ những điểm số đó).

Như vậy trong một chương trình máy tính, giải thuật và cấu trúc dữ liệu có mối quan hệ chặt chẽ với nhau, được thể hiện qua công thức :

Cấu trúc dữ liệu + Giải thuật = Chương trình

Với một cấu trúc dữ liệu đã chọn, sẽ có những giải thuật tương ứng, phù hợp. Khi cấu trúc dữ liệu thay đổi thường giải thuật cũng phải thay đổi theo để tránh việc xử lý gượng ép, thiếu tự nhiên trên một cấu trúc không phù hợp. Hơn nữa, một cấu trúc dữ liệu tốt sẽ giúp giải thuật xử lý trên đó có thể phát huy tác dụng tốt hơn, vừa đáp ứng nhanh vừa tiết kiệm bộ nhớ, giải thuật cũng dễ hiểu và đơn giản hơn.

Cấu trúc lưu trữ (Storage structures): CTDL được biểu diễn trong bộ nhớ máy tính còn được gọi là Cấu trúc lưu trữ



1.3 Phân tích thuật toán

1.3.1 Sự cần thiết phải phân tích giải thuật

Trong khi giải một bài toán chúng ta có thể có một số giải thuật khác nhau, vấn đề là cần phải đánh giá các giải thuật đó để lựa chọn một giải thuật tốt nhất. Thông thường thì ta sẽ căn cứ vào các tiêu chuẩn sau:

- (1) - Giải thuật đúng đắn.
- (2) - Giải thuật đơn giản.
- (3) - Giải thuật hiệu quả.

Với yêu cầu (1), để kiểm tra tính đúng đắn của giải thuật chúng ta có thể cài đặt giải thuật đó và cho thực hiện trên máy với một số bộ dữ liệu mẫu rồi lấy kết quả thu được so sánh với kết quả đã biết. Thực ra thì cách làm này không chắc chắn bởi vì có thể giải thuật đúng với tất cả các bộ dữ liệu chúng ta đã thử nhưng lại sai với một bộ dữ liệu nào đó. Và lại cách làm này chỉ phát hiện ra giải thuật sai chứ chưa chứng minh được là nó đúng. Tính đúng đắn của giải thuật cần phải được chứng minh bằng toán học. Tất nhiên điều này không đơn giản và do vậy chúng ta sẽ không đề cập đến ở đây.

Khi chúng ta viết một chương trình để sử dụng một vài lần thì yêu cầu (2) là quan trọng nhất. Chúng ta cần một giải thuật để viết chương trình để nhanh chóng có được kết quả, thời gian thực hiện chương trình không được đề cao vì dù sao thì chương trình đó cũng chỉ sử dụng một vài lần mà thôi.

Khi một chương trình được sử dụng nhiều lần thì yêu cầu tiết kiệm thời gian thực hiện chương trình, tiết kiệm không gian lưu trữ lại rất quan trọng, đặc biệt đối với những chương trình mà khi thực hiện cần dữ liệu nhập lớn do đó yêu cầu (3) sẽ được xem xét một cách kỹ càng. Ta gọi nó là hiệu quả của giải thuật. Tính hiệu quả thể hiện qua hai mặt:

- Thời gian (Giúp chương trình chạy nhanh)
- Không gian (Làm cho chương trình bé chiếm ít bộ nhớ, tiết kiệm không gian)

1.3.2 Thời gian thực hiện của giải thuật

Thời gian thực hiện của giải thuật phụ thuộc vào nhiều yếu tố

- Trước hết phụ thuộc vào độ lớn của dữ liệu đầu vào
- Ngoài ra T còn phụ thuộc vào
 - Máy
 - Ngôn ngữ
 - Kỹ xảo của người lập trình, ...

Vậy có cách nào đo được thời gian của thuật toán mà không phụ thuộc vào cấu hình máy tính, phần mềm và ngôn ngữ lập trình? Chúng ta xem xét cách tính thời gian thực hiện giải thuật $T(n)$, với n là kích thước của bài toán.

Thời gian thực hiện thuật toán là số các phép toán cơ bản thực hiện thuật toán đó, ký hiệu $T(n)$ trong đó n là kích thước (độ lớn) của dữ liệu vào. Các cách tính thời gian của thuật toán. Trong một số trường hợp chúng ta cần tính thời gian trung bình thực hiện giải thuật trung bình = (thời gian thực hiện trong trường hợp xấu nhất + Thời gian thực hiện trong thời gian tốt nhất) / 2

* Một số ví dụ đánh giá độ phức tạp tính toán của GT

Ví dụ 1.1 : Tính trung bình cộng của một dãy gồm n số nhập vào từ bàn phím

```
scanf("%d",&n); // so luong phan tu (1)
t = 0; // luu tong (2)
d = 1; //bien dem (3)
while (d <= n){ // (4)
    k = scanf("%d",&n); // (5)
    t = t+k; // (6)
    d++; // (7)
}
tbc= t/n; // (8)
printf("%d",tbc); // (9)
```

* Đánh giá giải thuật:

Bảng 1.1: Bảng tính độ phức tạp tính toán của thuật toán

Các lệnh	Số lần thực hiện
1	1
2	1
3	1
4	n
5	n
6	n
7	n
8	1
9	1
	$T(n) = 4n + 5$

Xét quan hệ giữa tài nguyên và độ lớn đầu vào, nếu như tìm được hằng số $C > 0$, không phụ thuộc vào n, sao cho với n đủ lớn, các hàm $T(n)$ và $g(n)$ đều dương và $T(n) \leq C.g(n)$ thì ta nói thuật toán có độ phức tạp $O(g(n))$.

Độ phức tạp không phải là độ đo chính xác lượng tài nguyên máy cần dùng, mà đặc trưng cho động thái của hệ thống khi kích thước đầu vào tăng lên. Chẳng hạn với thuật toán có độ phức tạp tuyến tính $O(n)$, nếu kích thước đầu vào tăng gấp đôi thì có thể ước tính rằng tài nguyên cần dùng cũng tăng khoảng gấp đôi. Nhưng với thuật toán có độ phức tạp bình phương $O(n^2)$ thì tài nguyên sẽ tăng gấp bốn. Mặt khác, với thuật toán có

độ phức tạp hàm mũ $O(2^n)$ thì chỉ cần cộng thêm 2 đơn vị vào độ lớn đầu vào cũng đã làm tài nguyên tăng gấp 4 lần (tức là theo cấp số nhân) rồi.

Các độ phức tạp thường gặp đối với các thuật toán thông thường gồm có:

Độ phức tạp hằng số, $O(1)$. Số phép tính/thời gian chạy/dung lượng bộ nhớ không phụ thuộc vào độ lớn đầu vào. Chẳng hạn như các thao tác hệ thống: đóng, mở Tập tin.

Độ phức tạp tuyến tính $O(n)$. Số phép tính/thời gian chạy/dung lượng bộ nhớ có xu hướng tỉ lệ thuận với độ lớn đầu vào. Chẳng hạn như tính tổng các phần tử của một mảng một chiều.

Độ phức tạp đa thức $O(P(n))$, với P là đa thức bậc cao (từ 2 trở lên). Chẳng hạn như các thao tác tính toán với mảng nhiều chiều (tính định thức ma trận).

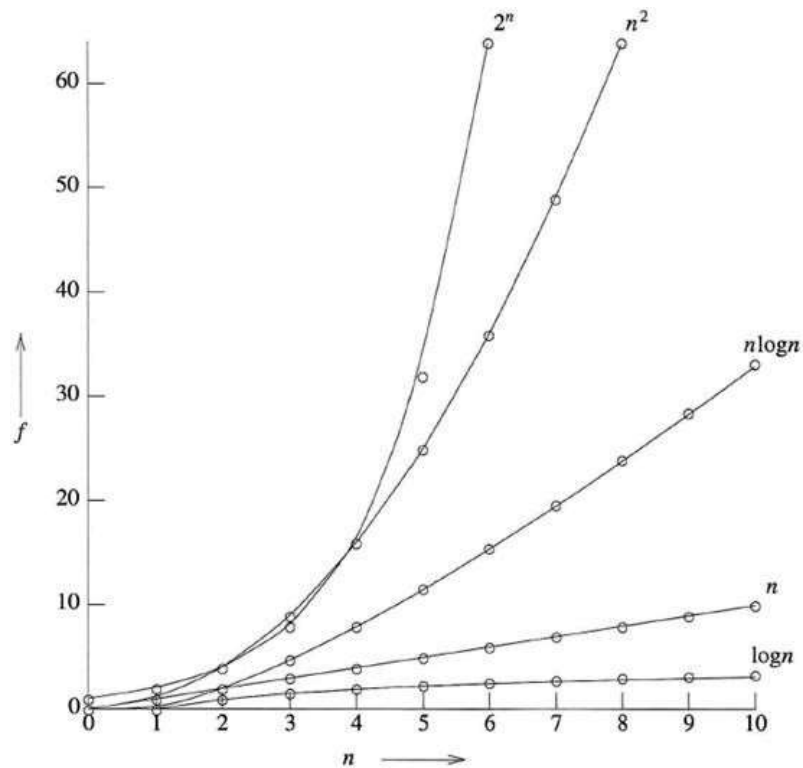
Độ phức tạp logarit $O(\log n)$ (chú ý: bậc của nó thấp hơn so với $O(n)$). Chẳng hạn thuật toán Euclid để tìm ước số chung lớn nhất.

Độ phức tạp hàm mũ $O(2^n)$. Trường hợp này bất lợi nhất và sẽ rất phi thực tế nếu thực hiện thuật toán với độ phức tạp này.

Bảng dưới đây mô tả giá trị của từng hàm khi n thay đổi:

Bảng 1.2: Độ phức tạp tính toán

$\log n$	n	$n \log n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65,536
5	32	160	1024	32,768	4,294,967,296



Hình 1.7: Biểu đồ thể hiện độ phức tạp tính toán

* Một số quy tắc cơ bản xác định độ phức tạp tính toán

a. Quy tắc loại bỏ hằng số

Nếu ta có $T(n) = O(C1.g(n))$ thì ta cũng có $T(n) = O(g(n))$

b. Quy tắc cộng

$T1(n)$ và $T2(n)$ là thời gian thực hiện của 2 đoạn chương trình $P1$ và $P2$ và $T1(n) = O(f(n))$; $T2(n) = O(g(n))$ thì thời gian thực hiện 2 đoạn chương trình nối tiếp nhau là $T(n) = T1(n) + T2(n)$ hay $T(n) = O(\max(f(n), g(n)))$

c. Quy tắc nhân

Thời gian thực hiện hai đoạn chương trình $P1$, $P2$ lồng nhau là: $T(n) = O(f(n).g(n))$

Tóm lại: Quy tắc tổng quát để phân tích một chương trình:

- Thời gian thực hiện của mỗi lệnh gán là $O(1)$

- Thời gian thực hiện của một chuỗi tuần tự các lệnh được xác định bằng quy tắc cộng. Như vậy thời gian này là thời gian thi hành một lệnh nào đó lâu nhất trong chuỗi lệnh.

- Thời gian thực hiện cấu trúc if là thời gian lớn nhất thực hiện lệnh if hoặc sau else và thời gian kiểm tra điều kiện. Thường thời gian kiểm tra điều kiện là $O(1)$.

- Thời gian thực hiện vòng lặp là tổng thời gian thực hiện thân vòng lặp (trên tất cả các lần lặp). Nếu thời gian thực hiện thân vòng lặp không thay đổi thì thời gian thực hiện vòng lặp là tích của số lần lặp với thời gian thực hiện 1 lần thân vòng lặp.

Ví dụ 1.2: Tính thời gian thực hiện của thuật toán sắp xếp chọn

```
void selectionSort(int* a[], int n){
    for(i=0; i<n; i++){
        k=i;
        for(j=i+1; j < n; j++){
            if(a[j]<a[k]){ // (*)
                k=j;
            }
        }
        temp=a[k];
        a[k]=a[i];
        a[i]=temp;
    }
}
```

*** Xác định độ phức tạp của giải thuật trên**

Ta thấy toàn bộ chương trình chỉ gồm một lệnh lặp {1}, lồng trong lệnh {1} là lệnh {2}, lồng trong lệnh {2} là lệnh {3} và lồng trong lệnh {3} là 3 lệnh nối tiếp nhau {4}, {5} và {6}. Chúng ta sẽ tiến hành tính độ phức tạp theo thứ tự từ trong ra:

- Trước hết, cả ba lệnh gán {4}, {5} và {6} đều tốn $O(1)$ thời gian, việc so sánh $a[j-1] > a[j]$ cũng tốn $O(1)$ thời gian, do đó lệnh {3} tốn $O(1)$ thời gian.

Vòng lặp {2} thực hiện $(n-i)$ lần, mỗi lần $O(1)$ do đó vòng lặp {2} tốn $O((n-i).1) = O(n-i)$.

Vòng lặp {1} lặp có i chạy từ 1 đến n-1 nên thời gian thực hiện của vòng lặp {1} và cũng là độ phức tạp của giải thuật là:

$$T(n) = \sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2} = O(n^2)$$

Chú ý:

Trong trường hợp vòng lặp không xác định được số lần lặp thì chúng ta phải lấy số lần lặp trong trường hợp xấu nhất.

1.3.3 Độ phức tạp không gian

Không gian của thuật toán, ký hiệu $L(gt)$ được tính bằng số ô nhớ cơ bản được sử dụng trong thuật toán đó

Ví dụ: Xét các bước của thuật toán tính trung bình cộng của một dãy gồm n số nhập vào từ bàn phím như trên (gt1). Khi đó $L(gt1)$ = tổng số byte lưu trữ của 5 biến cơ bản n, biến d, biến T, biến tbc và biến k.

1.4 Ngôn ngữ diễn đạt giải thuật

Mặc dù vấn đề ngôn ngữ lập trình không được đặt ra trong bài giảng này, nhưng vấn đề diễn đạt các thuật toán và thuận lợi cho việc cài đặt chúng ta cần phải lựa chọn một ngôn ngữ lập trình cụ thể, trong cuốn giáo trình này chúng tôi lựa chọn ngôn ngữ lập trình C. Tuy nhiên, về bản chất các cấu trúc dữ liệu và thuật toán là độc lập với ngôn ngữ lập trình, chúng ta có thể diễn đạt thuật toán và cấu trúc dữ liệu bằng ngôn ngữ lập trình mà chúng ta ưa thích (ví dụ: Pascal, C++, C#, Java, Python, ...).

Ngôn ngữ diễn đạt thuật toán là công cụ trung gian giúp giao tiếp giữa người và máy tính điện tử. Mỗi ngôn ngữ lập trình có một hệ kiểu, trong đó có một số là kiểu dữ liệu đơn hay nguyên tử, một số là các cấu trúc dữ liệu chứa các kiểu đơn/cấu trúc khác. Ngôn ngữ diễn đạt thuật toán bao gồm một tập hợp các câu lệnh tuân theo một cú pháp nhất định. Thông qua các câu lệnh mà máy tính có thể hiểu và thực hiện những công việc mà người dùng muốn máy tính làm.

1.5 Độ quy và giải thuật đệ quy

1.5.1 Khái niệm đệ quy

Đệ quy là một kỹ thuật dùng để định nghĩa một khái niệm trực tiếp hoặc gián tiếp theo chính nó. Một đối tượng được gọi là đệ quy nếu nó bao gồm chính nó như một bộ phận

hoặc nó được định nghĩa dưới dạng của chính nó; ví dụ: cho n là số nguyên dương thì giai thừa của n chính là một đối tượng đệ quy, $n! = (n - 1)! * n$.

1.5.2 Thuật toán đệ quy

Nếu lời giải của bài toán T được thực hiện bởi lời giải của bài toán T' có dạng như T thì đó là một lời giải đệ quy. Giải thuật chứa lời giải đệ quy gọi là giải thuật đệ quy ($T' < T$)

Ví dụ 1.3: thuật toán tìm số Fibonacci thứ n là một thuật toán đệ quy. Dãy số Fibonacci được định nghĩa như sau:

$$fib(n) = \begin{cases} 1 & ; n = 1, 2 \\ fib(n-1) + fib(n-2) & ; n > 2 \end{cases}$$

Chương trình con đệ quy là chương trình con trong đó có chứa lời gọi tới chính nó. Ví dụ về chương trình con đệ quy cho bài toán tìm số Fibonacci thứ n :

```
int fib(int n)
{
    if((n==1) || (n==2)){           // (1)
        return 1;                   // (2)
    } else{
        return (fib(n-1)+fib(n-2)); // (3)
    }
}
```

1.5.3 Đặc điểm của thuật toán đệ quy

- Trong thuật toán đệ quy bao giờ cũng có lời gọi đến chính tên thuật toán.
- Mỗi lần có lời gọi thì kích thước của bài toán thu nhỏ hơn trước.
- Có một trường hợp đặc biệt, trường hợp suy biến: Bài toán sẽ được giải quyết theo một cách khác hẳn và lời gọi đệ quy cũng kết thúc.

Trong ví dụ 1.3, các đặc điểm này được thể hiện như sau:

- Lời gọi đến chính nó (tên của thuật toán): câu lệnh (3).
- Mỗi lần có lời gọi thì kích thước của bài toán thu nhỏ hơn trước: Tại câu lệnh (3), kích thước bài toán được truyền vào lời gọi nhỏ hơn trước 1 và 2 đơn vị.
- Trường hợp suy biến: Lệnh (1).

Có 2 loại đệ quy, đệ quy trực tiếp (thủ tục chứa lời gọi đến chính nó) và đệ quy gián tiếp (thủ tục chứa lời gọi đến thủ tục khác mà thủ tục này lại chứa lời gọi đến chính nó). Phương pháp đệ quy có ưu điểm là rõ ràng, chặt chẽ, thiết kế thuật toán đơn giản. Tuy nhiên nó cũng có những hạn chế là lời gọi đệ quy tốn rất nhiều thời gian, dễ phát sinh chạy vô hạn.

1.6 Thuật toán sắp xếp

1.6.1 Bài toán sắp xếp

Sắp xếp là quá trình xử lý một danh sách các phần tử (hoặc các mẫu tin) để đặt chúng theo một thứ tự thỏa mãn một tiêu chuẩn nào đó dựa trên nội dung thông tin lưu giữ tại mỗi phần tử. Bài toán sắp xếp là một bài toán quan trọng, phổ biến đặc biệt là trong khoa học máy tính. Để tạo điều kiện cho việc tìm kiếm thông tin được nhanh và hiệu quả, bước then chốt, quan trọng đầu tiên trong nhiều thuật toán tìm kiếm đòi hỏi dữ liệu phải được sắp xếp theo một trình tự nhất định nào đó.

Hiện nay có nhiều giải thuật sắp xếp như: Selection sort, Insertion sort, Bubble sort, Binary Insertion sort, Shell sort, Heap sort, Quick sort, Merge sort, Radix sort. . .

1.6.2 Một số thuật toán sắp xếp cơ bản

1.6.2.1 Thuật toán sắp xếp chọn (Selection Sort)

Sắp xếp chọn là một thuật toán sắp xếp tương đối dễ hiểu và đơn giản.

a) Ý tưởng

Cho 1 dãy gồm n phần tử bất kỳ từ $a[0]$ đến $a[n-1]$. Ta tiến hành xét từ phần tử đầu tiên $a[0]$ của dãy, chọn phần tử có khóa nhỏ nhất trong n phần tử từ $a[0]$ đến $a[n-1]$ và hoán vị nó với phần tử $a[0]$ ta được phần tử đầu tiên có vị trí đúng. Bỏ qua phần tử vừa được xét, tiếp tục chọn phần tử có khóa nhỏ nhất trong $n-1$ phần tử từ $a[1]$ đến $a[n-1]$ rồi hoán vị nó với $a[1]$. Lặp lại quá trình trên cho dãy hiện hành đến khi dãy hiện thời chỉ còn lại 1 phần tử.

Ở bước thứ i , chọn phần tử có khóa nhỏ nhất trong $n - i + 1$ phần tử từ $a[i-1]$ đến $a[n-1]$ rồi hoán vị nó với $a[i-1]$. Cứ như vậy sau $n - 1$ bước thì mảng đã được sắp xếp theo thứ tự.

Các bước sắp xếp

Bước 1: $i = 0$ // lần xử lý đầu tiên

Bước 2: Tìm phần tử nhỏ nhất $a[\min]$ trong dãy hiện hành từ $a[i]$ đến $a[n - 1]$

Bước 3: Hoán vị $a[\min]$ và $a[i]$

Bước 4: Nếu $i < n - 1$ thì $i = i + 1$; Lặp lại Bước 2

Ngược lại: Dừng

b) Chương trình:

```
void selection_Sort(int a[], int n)
{
    int i, j, vtmin;
    for(i=0; i<n-1;i++)
    {
        vtmin = i;
        for (j=i+1, j<n, j++)
            if (a[j]< a[vtmin])
                vtmin = j;
        swap(a[vtmin], a[i]);
    }
}
```

Ví dụ 1.4: Cho dãy số ban đầu gồm các số: 5 6 2 2 10 12 9 10 9 3

Các bước thực hiện sắp xếp như sau:

Bước 1: 2 | 6 5 2 10 12 9 10 9 3

Bước 2: 2 2 | 5 6 10 12 9 10 9 3

Bước 3: 2 2 3 | 6 10 12 9 10 9 5

Bước 4: 2 2 3 5 | 10 12 9 10 9 6

Bước 5: 2 2 3 5 6 | 12 9 10 9 10

Bước 6: 2 2 3 5 6 9 | 12 10 9 10

Bước 7: 2 2 3 5 6 9 9 | 10 12 10

Bước 8: 2 2 3 5 6 9 9 10 | 12 10

Bước 9: 2 2 3 5 6 9 9 10 10 | 12

Kết quả: 2 2 3 5 6 9 9 10 10 12

c) Đánh giá:

Các lệnh gán lấy $O(1)$ thời gian, Swap = $O(1)$ thời gian, vòng lặp for thực hiện $n - i$ lần (j chạy từ $i + 1$ tới n) mỗi lần lấy $O(1)$. Vậy lấy $O(n - i)$ thời gian. Vậy thời gian tính toán là:

$$T(n) = \sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2} = O(n^2)$$

1.6.2.2 Thuật toán sắp xếp chèn (Insertion Sort)

Một trong những thuật toán sắp xếp đơn giản nhất là thuật toán sắp xếp chèn. Ví dụ về một loại sắp xếp chèn được ứng dụng trong cuộc sống hàng ngày là trong khi chơi bài. Để sắp xếp các thẻ trong tay, bạn rút thẻ, thay đổi các thẻ còn lại, và sau đó chèn thẻ rút ra vào đúng chỗ. Quá trình này được lặp lại cho đến khi tất cả các thẻ nằm trong dãy đã được sắp xếp theo đúng thứ tự.

a) Ý tưởng: Khởi đầu dãy đích (dãy đã sắp) là rỗng. Lấy dần từng phần tử từ dãy nguồn (dãy chưa sắp), chèn vào vị trí thích hợp trong dãy đã sắp. Công việc này được lặp lại cho đến khi dãy ban đầu trở thành rỗng. Ta thu được dãy đích là dãy kết quả của bài toán. Giả sử xem phần tử $a[0]$ là một dãy đã có thứ tự (đã sắp):

Bước 1: Chèn phần tử $a[1]$ vào danh sách đã có thứ tự $a[0]$ sao cho $a[0], a[1]$ là một danh sách có thứ tự.

Bước 2: Chèn phần tử $a[2]$ vào danh sách đã có thứ tự $a[0], a[1]$ sao cho $a[0], a[1], a[2]$ là một danh sách có thứ tự, . . .

Bước i: Chèn phần tử $a[i]$ vào danh sách đã có thứ tự $a[0], a[1], \dots, a[i - 1]$ sao cho $a[0], a[1], \dots, a[i]$ là một danh sách có thứ tự.

Phần tử đang xét $a[j]$ sẽ được chèn vào vị trí thích hợp trong danh sách các phần tử đã được sắp trước đó $a[0], a[1], \dots, a[j - 1]$ bằng cách so sánh $a[j]$ với $a[j - 1]$ đứng ngay trước nó. Nếu $a[j] < a[j - 1]$ thì đổi chỗ $a[j]$ với $a[j - 1]$ và tiếp tục so sánh $a[j - 1]$ với $a[j - 2]$. . . Lặp cho đến khi hết dãy ($i = n - 1$) như vậy mảng được sắp xếp xong.

b) Chương trình

```

void insertion_Sort(int a[], int n)
{
    int i, j, temp;
    for(i=0; i<n-1; i++)
    {
        int j = i;
        temp = a[i];
        while(j>0 &&temp < a[j-1])
        {
            a[j] = a[j-1];
            j = j-1;
        }
        a[j] = temp;
    }
}

```

Ví dụ 1.5: Cho dãy số ban đầu gồm các số: 5 6 2 2 10 12 9 10 9 3

Bước 1 : 5 6

Bước 2: 2 5 6

Bước 3: 2 2 5 6

Bước 4: 2 2 5 6 10

Bước 5: 2 2 5 6 10 12

Bước 6: 2 2 5 6 9 10 12

Bước 7: 2 2 5 6 9 10 10 12

Bước 8: 2 2 5 6 9 9 10 10 12

Bước 9: 2 2 3 5 6 9 9 10 10 12 (Kết quả)

c) Đánh giá giải thuật

Độ phức tạp giải thuật phụ thuộc vào số lần so sánh và gán các phần tử. Ở bước thứ i, tối đa cần i lần so sánh để tìm được vị trí chèn thích hợp. Do vậy số lần so sánh tối đa là:

$$\sum_{i=1}^{n-1} i = n(n-1)/2$$

Số phép gán giá trị phần tử: $n(n-1) + 2n$ Số phép gán chỉ số: $n(n-1)$

Vì vậy độ phức tạp là tương đương $O(n^2)$

1.6.2.3 Thuật toán sắp xếp nổi bọt (Bubble Sort)

a) Ý tưởng: Xuất phát từ cuối dãy, đổi chỗ các cặp phần tử cạnh nhau để đưa phần tử nhỏ hơn trong cặp phần tử đó về vị trí đứng đầu dãy hiện hành, sau đó sẽ không xét đến nó ở bước tiếp theo, do vậy ở lần xử lý thứ i sẽ có vị trí đầu dãy là i . Lặp lại quá trình trên cho đến khi không còn cặp phần tử nào để xét. Cụ thể:

Bước 1: $i = 0$ // lần xử lý đầu tiên

Bước 2: $j = N - 1$ // duyệt từ cuối dãy

Trong khi $j > i$ thực hiện:

Nếu $a[j] < a[j - 1]$: hoán vị $a[j]$ và $a[j - 1]$ $j = j - 1$

Bước 3: $i = i + 1$ // lần xử lý tiếp theo

Nếu $i > N - 1$ thì dừng

Ngược lại, lặp lại Bước 2

Sau n bước ta thu được mảng đã được sắp xếp theo đúng thứ tự

b) Ví dụ:

Ban đầu	Bước 1	Bước 2	Bước 3	Bước 4
5	2	2	2	2
6	5	2	2	2
2	6	5	3	3
2	2	6	5	5
10	3	3	6	6
12	10	9	9	9
9	12	10	9	9
10	9	12	10	10
9	10	9	12	10
3	9	10	10	12

c) Chương trình

```

void bubble_Sort(int a[], int n)
{
    int i, j;
    for(i=0; i<=n-1; i++)
        for(j=n; j>=i; j--)
            if(a[j-1]>a[j])
                swap(a[j-1], a[j]);
}

```

d) Đánh giá giải thuật:

Ở lượt thứ i , bao giờ cũng cần $(n - i + 1)$ lần so sánh để xác định phần tử nhỏ nhất hiện hành. Do vậy số lần so sánh là:

$$\sum_{i=1}^{n-1} (n - i + 1) = n(n - 1)/2$$

Vậy độ phức tạp của thuật toán là $O(n^2)$

1.7 Thuật toán tìm kiếm

Trong hầu hết các hệ thống tin học, khi khai thác, xử lý dữ liệu chúng ta đều phải thực hiện thao tác tìm kiếm và xử lý thông tin.

Chẳng hạn trong hệ thống tra cứu điểm thi, việc tra cứu, tìm kiếm kết quả thi của thí sinh diễn ra rất nhanh chóng và chính xác. Thí sinh chỉ cần truy cập vào hệ thống tra cứu điểm thi, rồi nhập thông tin của mình như họ tên, ngày tháng năm sinh hoặc số báo danh là hệ thống nhanh chóng đưa ra kết quả thi của thí sinh đó mà không cần tìm đến trường mình dự thi để tìm hiểu kết quả thi của mình.

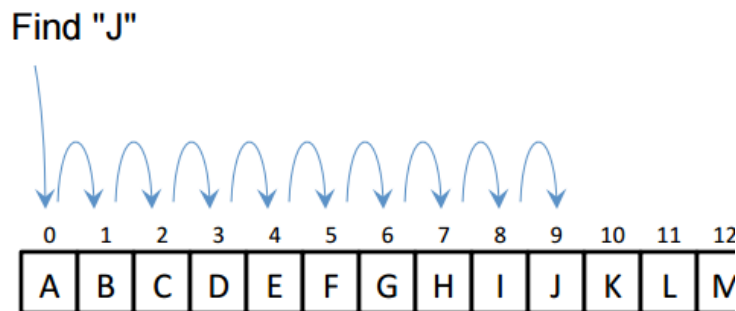
Đây là một trong rất nhiều ứng dụng của bài toán tìm kiếm trong các hệ thống tin học. Chúng ta sẽ đi tìm hiểu về bài toán tìm kiếm và nghiên cứu đánh giá các thuật giải tìm kiếm để đưa ra được giải thuật phù hợp nhất với yêu cầu của bài toán đặt ra.

1.7.1 Tìm kiếm tuần tự (tuyến tính)

Tìm kiếm tuyến tính (linearsearch) hay tìm kiếm tuần tự (sequentialsearch) là phương pháp tìm kiếm một phần tử cho trước trong một danh sách bằng cách duyệt từng phần tử của danh sách đó cho đến khi tìm thấy phần tử cần tìm hay đã duyệt qua toàn bộ danh sách. Tại mỗi bước, khoá của danh sách sẽ được so sánh với giá trị cần tìm. Quá trình tìm

kiểm kết thúc khi đã tìm thấy khoá thoả mãn hoặc đã duyệt hết danh sách.

Ví dụ 1.6: Cho chuỗi ký tự gồm các phân tử: A, B, C, D, ... Tìm xem phân tử $x = j$ có trong chuỗi ký tự hay không?



Ví dụ 1.7: cho danh sách A gồm các phân tử: 5, 18, 20, 7, 10, 15, 19. Tìm xem phân tử $x = 7$ có trong danh sách hay không. Chúng ta lần lượt so sánh phân tử $x = 7$ với các phân tử của danh sách và tìm được phân tử $x = 7$ tại vị trí thứ 4 của danh sách.

Tìm phân tử $x = 25$ có trong danh sách hay không. Cũng tương tự như trên ta lần lượt so sánh phân tử $x = 25$ với các phân tử có trong danh sách, và với ví dụ này ta duyệt đến cuối danh sách cũng không tìm thấy phân tử nào có giá trị $x = 25$. Kết luận, phân tử $x = 25$ không có trong danh sách.

Các bước thực hiện giải thuật:

Đầu vào: mảng $a[]$ gồm n phân tử và giá trị x cần tìm. Đầu ra: Trả về True nếu tìm thấy, ngược lại trả về False

1. Bước 1: $i = 0$; // bắt đầu từ phân tử đầu tiên của dãy
2. Bước 2: So sánh $a[i]$ với x , có 2 khả năng: Nếu $a[i] = x$: Tìm thấy dừng Ngược lại, chuyển sang Bước 3.
3. Bước 3: $i = i + 1$; // xét tiếp phân tử kế trong mảng

Cài đặt:

```

int linearSearch(int a[], int N, int x)
{
    int i=0;
    while ((i<N) && (a[i]!=x))
        i++;
    if(i==N)
        return -1; //tim het mang
    else
        return i; //a[i] la phan tu co khoa x
}

```

Thủ tục này tiến hành duyệt từ đầu mảng. Nếu tại vị trí nào đó, giá trị của phần tử bằng với giá trị cần tìm thì hàm trả về chỉ số tương ứng của phần tử trong mảng. Nếu không tìm thấy giá trị trong toàn bộ mảng thì hàm trả về giá trị là -1 .

Độ phức tạp thời gian: $O(n)$ khi phần tử tìm kiếm nằm cuối danh sách hoặc không có trong danh sách

Thời gian chạy tốt nhất: $O(1)$ khi phần tử cần tìm nằm ngay đầu danh sách.

1.7.2 Tìm kiếm nhị phân

Thuật toán tìm kiếm nhị phân là một thuật toán dùng để tìm kiếm phần tử trong một danh sách đã được sắp xếp (tìm kiếm kiểu “tra từ điển”). Thuật toán hoạt động như sau: trong mỗi bước, so sánh phần tử cần tìm với phần tử nằm ở chính giữa danh sách. Nếu hai phần tử bằng nhau thì phép tìm kiếm thành công và thuật toán kết thúc. Nếu chúng không bằng nhau thì kiểm tra xem phần tử cần tìm nhỏ hơn hay lớn hơn phần tử ở giữa danh sách. Nếu nhỏ hơn thì thuật toán lặp lại bước so sánh trên với nửa đầu của danh sách, còn nếu phần tử cần tìm lớn hơn phần tử giữa thì tiến hành lặp lại bước so sánh với nửa sau của danh sách. Sau mỗi bước số lượng phần tử trong danh sách cần xem xét giảm đi một nửa, nên thời gian thực thi của thuật toán là $O(\log n)$.

Các bước thực hiện như sau:

Bước 1: $\text{lef } t = 0; \text{ right} = N - 1$; // tìm kiếm trên tất cả các phần tử

Bước 2: $\text{mid} = (\text{lef } t + \text{right})/2$; // lấy mốc so sánh So sánh $a[\text{mid}]$ với x , có 3 khả năng:

+ $a[\text{mid}] = x$: Tìm thấy. Dừng lại.

+ $a[\text{mid}] > x$: // tìm tiếp x trong dãy con $a(\text{left}) \dots a(\text{mid}-1)$: $\text{right} = \text{mid} - 1$;

+ a[mid] < x: //tìm tiếp x trong dãy a(mid+1)...a(right): left = mid + 1;

Bước 3: Nếu left <= right //còn phần tử chưa xét=> tìm tiếp

Lặp lại Bước 2

Ngược lại: Dừng; //Đã xét hết phần tử

Cài đặt:

```
int binarySearch(int a[], int N, int x)
{
    int left = 0, right = N - 1;
    int mid;
    do
    {
        mid = (left + right) / 2;
        if (x == a[mid])
            return mid; //Thay x tại mid
        else if (x < a[mid])
            right = mid - 1;
        else
            left = mid + 1;
    }
    while (left <= right);
    return -1; // tìm hết dãy mà không có x
}
```

Đánh giá thuật toán tìm kiếm nhị phân:

Độ phức tạp trung bình: $O(\log n)$

Độ phức tạp tốt nhất: $O(1)$

Một số ưu nhược điểm giữa hai thuật toán với nhau:

- Tìm kiếm nhị phân chỉ hoạt động trên mảng đã sắp xếp có thứ tự
- Tìm kiếm nhị phân phù hợp với mảng có độ dài lớn
- Thuật toán tìm kiếm nhị phân có hiệu suất tốt hơn
- Tìm kiếm tuần tự phù hợp với mảng nhỏ và chưa sắp xếp.

1.8 Thực hành

1.8.1 Mục đích của bài thực hành

Bài thực hành giúp sinh viên hiểu về tư tưởng một số thuật toán sắp xếp, tìm kiếm cơ bản, giải thuật đệ quy. Sau đó, sinh viên thực hành thành thạo trên các phép toán sắp xếp, tìm kiếm và giải thuật đệ quy. Cuối cùng, sinh viên có khả năng vận dụng vào triển khai bài toán ứng dụng.

1.8.2 Bài thực hành mẫu

Bài 1: Xây dựng giải thuật đệ quy tính giai thừa của một số nguyên

```
#include<stdio.h>
int giai thua(int n){
    if(n==0)return 1;
    return n * giai thua(n-1);
}
int main(){
    int n;
    printf("Nhap gia tri n = ");
    scanf("%d",&n);
    printf("Giai thua cua %d la: %d",giai thua(n));
    return 0;
}
```

Bài 2: Xây dựng chương trình nhập vào 1 mảng n số nguyên. Sắp xếp mảng theo thuật toán Bubble sort và đưa kết quả ra màn hình

```
#include <stdio.h>
void swap(int &x, int &y)
{
    int temp = x;
    x = y;
    y = temp;
}
// Hàm sap xep bubble sort
void bubbleSort(int arr[], int n)
{
    int i, j;
    bool haveSwap = false;
    for (i = 0; i < n-1; i++){
        // i phan tu cuoi cung da duoc sap xep
        haveSwap = false;
```

```

        for (j = 0; j < n-i-1; j++){
            if (arr[j] > arr[j+1]){
                swap(arr[j], arr[j+1]);
                haveSwap = true; // Kiểm tra lần này có swap không
            }
        }
        // Nếu không có phần tử nào cần swap thì mảng đã được sắp xếp
        if(haveSwap == false){
            break;
        }
    }
}
/* Hàm xuất mảng */
void printArray(int arr[], int size)
{
    int i;
    for (i=0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
int main()
{
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr)/sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}

```

1.8.3 Bài tập luyện tập

Bài 3: Xây dựng chương trình nhập vào 1 mảng n số nguyên. Sắp xếp mảng theo thuật toán Insertion sort và đưa kết quả ra màn hình.

Bài 4: Xây dựng chương trình nhập vào 1 mảng n số nguyên. Sắp xếp mảng theo thuật toán Selection sort và đưa kết quả ra màn hình.

Bài 5: Xây dựng chương trình tìm số Fibonacci thứ n bằng giải thuật đệ quy.

Bài 6: Xây dựng chương trình tìm kiếm số nguyên x trong dãy n số nguyên bằng thuật toán tìm kiếm tuần tự (Linear search)

Bài 7: Xây dựng thuật toán đệ quy đảo ngược một chuỗi ký tự.

Bài 8: Xây dựng thuật toán đệ quy tìm ước chung lớn nhất của 2 số nguyên.

Bài 9: Xây dựng chương trình tìm kiếm một số trong một dãy n số nguyên bằng thuật toán tìm kiếm nhị phân (Binary search).

Bài 10: Xây dựng chương trình tìm vị trí đầu tiên xuất hiện số x trong một dãy n số nguyên bằng thuật toán tìm kiếm nhị phân (Binary search).